Infrastructure as Code (IaC) Kya Hota Hai?

Infrastructure as Code (IaC) ka matlab hai ki hum apne cloud resources (jaise Resource Group, Virtual Machine, Storage Account, Network, etc.) ko manually portal se create karne ke bajaye code ke through manage karte hain.

Isse:

Automation possible hoti hai.

Human errors kam hote hain.

Same infrastructure ko baar-baar easily create kiya ja sakta hai.

Version control (Git) me code maintain kiya ja sakta hai.

Infrastructure Manage Karne Ke 2 Approaches

1. Imperative Approach (Azure CLI)

Meaning

Imperative approach me aap system ko step-by-step instructions dete ho ki kya karna hai.

Simple language me:

"Kaise karna hai" ye batana padta hai.

Example

Agar Resource Group banana hai to process kuch is tarah hogi:

Azure CLI Install karo.

Azure account me login karo.

Resource Group create karne ki command run karo.

Example:

az login

az group create --name MyRG --location centralindia

Yahan hum har step manually bata rahe hain.

Real Life Example

Jaise kisi ko chai banana sikhani ho:

Gas on karo

Pani garam karo

Chai patti dalo

Doodh dalo

Sugar dalo

Har step batana pad raha hai.

Isi ko Imperative Approach kehte hain.

2. Declarative Approach (Terraform, Bicep, ARM, Pulumi)

Meaning

Declarative approach me hum ye nahi batate ki resource kaise banega.

Hum sirf ye batate hain ki final result kya chahiye.

Yani:

"Mujhe ye infrastructure chahiye."

Baaki kaam tool khud karta hai.

Real Life Example

Jaise architect ko bolna:

Mujhe 5 BHK house chahiye.

Aap ye nahi batate:

Kitni bricks lagenge

Cement kitna lagega

Electric wire kaise lagegi

Architect sab manage karta hai.

Terraform bhi exactly wahi karta hai.

IaC Tools Comparison

1. ARM Templates

ARM (Azure Resource Manager Templates) Microsoft ka purana IaC tool hai.

Problems

Syntax kaafi lengthy hota hai.

JSON format me likhna padta hai.

Read karna aur maintain karna difficult hota hai.

Isi wajah se kaafi log ise avoid karte hain.

2. Bicep

Bicep Microsoft ka modern IaC tool hai.

Advantages

ARM se easy hai.

Syntax simple hai.

Microsoft officially support karta hai.

Limitation

Sirf Azure ke liye use hota hai.

AWS ya GCP ke liye use nahi kar sakte.

3. Pulumi

Pulumi ek modern IaC tool hai.

Iski khas baat

Configuration language alag nahi seekhni padti.

Aap directly programming languages use kar sakte ho.

Jaise:

Java

Python

C#

TypeScript

Go

Developers ke liye kaafi useful hai.

4. Terraform (Industry Standard)

Terraform sabse popular aur industry me sabse zyada use hone wala IaC tool hai.

Ye HashiCorp ne develop kiya tha aur baad me IBM ne HashiCorp ko acquire kar liya.

Aaj ke time me almost har company Terraform use karti hai.

Terraform Deep Dive

Terraform Kya Hai?

Terraform ek Open Source Infrastructure as Code (IaC) tool hai.

Iski help se hum cloud infrastructure ko code ke through create, update aur delete kar sakte hain.

Terraform Ki Features

1. Written in Go

Terraform Go programming language me develop hua hai.

Isliye ye:

Fast hai

Lightweight hai

Stable hai

2. Multi-Cloud Support

Ye Terraform ki sabse badi strength hai.

Ek hi tool se aap multiple cloud platforms manage kar sakte ho.

Jaise:

Microsoft Azure

AWS

Google Cloud (GCP)

Basic Questions

1. What is Infrastructure as Code (IaC)?

Answer:

Infrastructure as Code (IaC) is the process of managing and provisioning infrastructure using code instead of manual configuration. It helps automate deployments, reduce human errors, and maintain consistency.

2. What are the benefits of Infrastructure as Code?

Answer:

Automation

Faster deployments

Consistency

Version control

Reduced human errors

Easy rollback

Reusable infrastructure

3. What is Terraform?

Answer:

Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp. It allows users to create, modify, and manage infrastructure across multiple cloud providers using code.

4. Why is Terraform so popular?

Answer:

Multi-cloud support

Open source

Easy HCL syntax

State management

Large community support

Reusable modules

5. Who developed Terraform?

Answer:

Terraform was developed by HashiCorp.

6. Which language is Terraform written in?

Answer:

Terraform is written in Go (Golang).

7. Which language does Terraform use for writing configurations?

Answer:

Terraform uses HCL (HashiCorp Configuration Language).

Intermediate Questions

8. What is the difference between Imperative and Declarative approaches?

Answer:

Imperative	Declarative
Tells the system how to perform tasks	Tells the system what the final state should be
Step-by-step instructions	Desired state definition
Example: Azure CLI	Example: Terraform

9. What is Terraform State?

Answer:

Terraform State is a file that stores information about the infrastructure managed by Terraform. It helps Terraform understand the current state and determine what changes are needed.

10. What is the default Terraform state file?

Answer:

terraform.tfstate

11. Why is Terraform State important?

Answer:

Tracks infrastructure

Detects changes

Prevents duplicate resource creation

Supports updates and deletions

12. What are Terraform Providers?

Answer:

Providers are plugins that allow Terraform to communicate with cloud platforms such as Azure, AWS, Google Cloud, Kubernetes, VMware, etc.

13. What is a Terraform Module?

Answer:

A Terraform Module is a reusable collection of Terraform configuration files used to organize and simplify infrastructure code.

14. What is the difference between Terraform and ARM Templates?

Answer:

Terraform ARM Template

Multi-cloud Azure only

Uses HCL Uses JSON

Easy to read Complex syntax

Open source Microsoft tool

15. What is the difference between Terraform and Bicep?

Answer:

Terraform Bicep

Multi-cloud Azure only

HCL language Bicep language

HashiCorp Microsoft

Command-Based Questions

16. What does terraform init do?

Answer:

It initializes the Terraform working directory and downloads the required providers.

17. What does terraform plan do?

Answer:

It shows what changes Terraform will make before applying them.

18. What does terraform apply do?

Answer:

It creates or updates infrastructure according to the Terraform configuration.

19. What does terraform destroy do?

Answer:

It deletes all infrastructure managed by Terraform.

20. What does terraform validate do?

Answer:

It checks whether the Terraform configuration is syntactically correct.

21. What does terraform fmt do?

Answer:

It automatically formats Terraform code according to standard style guidelines.

Scenario-Based Questions

22. Why would you choose Terraform over Azure CLI?

Answer:

Terraform is preferred for large-scale infrastructure because it is declarative, reusable, supports version control, maintains state, and works across multiple cloud providers. Azure CLI is more suitable for quick administrative tasks.

23. Can Terraform manage existing infrastructure?

Answer:

Yes. Existing infrastructure can be brought under Terraform management using the terraform import command.

24. Can Terraform work with multiple cloud providers?

Answer:

Yes. Terraform supports Azure, AWS, Google Cloud, Oracle Cloud, Kubernetes, VMware, and many other providers.

25. What file extension is used for Terraform configuration files?

Answer:

.tf

What is Terraform?

Terraform ek Open Source Infrastructure as Code (IaC) tool hai jo infrastructure ko code ke through create, update aur delete karne ke liye use hota hai.

Simple words me:

"Terraform ki help se hum Azure Portal me manually click karne ke bajaye code likhkar infrastructure create karte hain."

Example:

Resource Group

Virtual Machine (VM)

Storage Account

Virtual Network (VNet)

Database

Ye sab Terraform se automatically create kiye ja sakte hain.

How Terraform Works?

Terraform Infrastructure as Code (IaC) concept par kaam karta hai.

Working Process

Step 1: Write Configuration

Sabse pehle hum Terraform configuration ek .tf file me likhte hain.

Example:

main.tf

Is file me hum define karte hain ki hume kya infrastructure chahiye.

Step 2: Terraform Reads the File

Terraform .tf file ko read karta hai aur samajhta hai ki kaun-kaun se resources create karne hain.

Step 3: Terraform Connects to Azure

Terraform Azure Provider ke through Azure Cloud se connect hota hai.

Step 4: Infrastructure is Created

Terraform automatically Azure me resources create kar deta hai.

Example:

Resource Group

Storage Account

Virtual Machine

Virtual Network

Simple Flow

Write .tf File

Terraform Reads File

Terraform Installation (Windows) – Step by Step (Hinglish Notes)

Step 1: Open Terraform Official Website

Google me search karo:

Install Terraform

Ya directly official website open karo:

https://developer.hashicorp.com/terraform/downloads

Step 2: Download Terraform

* Windows select karo.

* **AMD64 (64-bit)** version download karo.

Example downloaded file:

terraform_1.14.6_windows_amd64.zip

Step 3: Extract the ZIP File

Download hone ke baad ZIP file par right-click karo.

Right Click → Extract All

Extract karne ke baad ek file milegi:

terraform.exe

Ye hi Terraform executable file hai.

Step 4: Create a Folder

C drive me ek folder bana lo.

Example:

C:\Terraform

Ab extracted **terraform.exe** ko is folder ke andar copy ya move kar do.

Final structure:

```
C:\
 │
 └── Terraform
      │
      └── terraform.exe
```

Step 5: Add Terraform Folder to Environment Variables (PATH)

Windows Search

Search karo:

Environment Variables

Open karo:

Edit the system environment variables

System Properties

Click:

Environment Variables

Path Select Karo

System Variables me:

Path

Select karo.

Click:

Edit

New Path Add Karo

Click:

New

Aur folder ka path add karo:

C:\Terraform

Save

Click:

OK

Phir:

OK

Phir:

OK

Environment Variable save ho jayegi.

Step 6: Restart PowerShell

Agar PowerShell pehle se open hai to usse close karo.

Naya **PowerShell** ya **Command Prompt** open karo.

Step 7: Verify Terraform Installation

PowerShell me command run karo:

```bash
terraform --version
```

Example Output:

```
Terraform v1.14.6
on windows_amd64
```

Iska matlab Terraform successfully install ho gaya hai.

Alternative Verification

```bash
terraform --help
```

Agar help menu open ho jaye to installation successful hai.

Example:

Usage: terraform [global options] <subcommand> [args]

Installation Flow

```
Download Terraform ZIP
        │
        ▼
Extract ZIP File
        │
        ▼
Get terraform.exe
        │
        ▼
Create C:\Terraform Folder
        │
        ▼
Move terraform.exe
        │
        ▼
Add C:\Terraform to PATH
        │
        ▼
Restart PowerShell
        │
        ▼
terraform --version
```

Common Errors & Solutions

Error 1:

```text
terraform is not recognized as an internal or external command
```

Reason: PATH set nahi hua ya PowerShell restart nahi kiya.

Solution:

* Check karo ki `terraform.exe` `C:\Terraform` me hai.

* Verify karo ki `C:\Terraform` `PATH` me add hai.

* PowerShell/Command Prompt ko close karke dubara open karo.

Error 2:

```text
The system cannot find the path specified
```

Reason: PATH me galat folder add kiya gaya.

Solution: PATH me exact folder (`C:\Terraform`) add karo, `terraform.exe` ka full file path nahi.

Interview Questions

Q1. How do you install Terraform on Windows?

Answer: Download the Terraform ZIP from the official website, extract it, move `terraform.exe` to a folder (e.g., `C:\Terraform`), add that folder to the system PATH, restart the terminal, and verify using `terraform --version`.

Q2. Why do we add Terraform to the PATH environment variable?

Answer: So that the `terraform` command can be executed from any directory without specifying the full path to `terraform.exe`.

Q3. How do you verify Terraform installation?

Answer: Run either:

```bash
terraform --version
```

or

```bash
terraform --help
```

Q4. Which file do you get after extracting the Terraform ZIP?

Answer: `terraform.exe`.

Terraform Language (HCL)

Terraform me infrastructure likhne ke liye HCL (HashiCorp Configuration Language) use hoti hai.

Full Form:

HCL = HashiCorp Configuration Language

HCL ko specially Infrastructure as Code (IaC) ke liye design kiya gaya hai. Iska syntax simple, readable aur easy to understand hota hai.

Example

provider "azurerm" {

features {}

}

Is code me hum Azure Provider define kar rahe hain.

Key Points

Terraform HCL language use karta hai.

HCL human-readable hoti hai.

Iska use cloud infrastructure define karne ke liye hota hai.

Terraform Registry

Terraform Registry Terraform ki official documentation website hai.

Yahan se aap:

Oracle Cloud

VMware

Kubernetes

Isliye ise Vendor Independent Tool bhi kaha jata hai.

3. State Management

Terraform ek State File maintain karta hai.

Ye file yaad rakhti hai:

Kaun-kaun se resources create hue hain

Kya update hua hai

Kya delete hua hai

Isse Terraform ko infrastructure ka current status pata rehta hai.

Isi feature ki wajah se unnecessary changes nahi hote.

4. Strong Community Support

Terraform open-source hai.

Isliye:

Bahut bada community support hai.

Thousands of modules available hain.

Documentation kaafi achhi hai.

Har cloud provider ka support milta hai.

Terraform Language

Terraform apni language use karta hai jiska naam hai:

HCL (HashiCorp Configuration Language)

Ye language specially Infrastructure define karne ke liye banayi gayi hai.

Iska syntax simple aur readable hota hai.

Example:

resource "azurerm_resource_group" "rg" {

name = "MyResourceGroup"

location = "Central India"

}

Yahan hum sirf batate hain ki:

Resource Group ka naam kya hoga

Location kya hogi

Terraform baaki ka kaam khud kar leta hai.

Terraform Configuration File

Terraform code generally .tf extension wali file me likha jata hai.

Examples:

main.tf

resource.tf

network.tf

storage.tf

chameli.tf

File ka naam kuch bhi ho sakta hai, bas extension .tf hona chahiye.

Terraform Working Process

Terraform ka basic workflow kuch is tarah hota hai:

Step 1: Terraform Install Karo

Sabse pehle apne system me Terraform install karo.

Step 2: Azure Login

Azure account authenticate karne ke liye login karo:

az login

Browser open hoga aur Azure account se login ho jaoge.

Step 3: Terraform Configuration Likho

Ek .tf file create karo.

Example:

main.tf

Usme Resource Group ya dusre Azure resources define karo.

Step 4: Terraform Commands Run Karo

Terraform configuration ko deploy karne ke liye commands chalayi jaati hain.

Common commands:

terraform init

Providers download karta hai aur project initialize karta hai.

terraform plan

Batata hai ki kaun-kaun se changes hone wale hain.

terraform apply

Infrastructure create ya update karta hai.

terraform destroy

Jo resources Terraform ne create kiye hain unhe delete kar deta hai.

Imperative vs Declarative (Quick Revision)

Imperative (Azure CLI) Declarative (Terraform)

Step-by-step batana padta hai Sirf desired state define karni hoti hai

"Kaise karna hai" "Kya chahiye"

Manual commands execute hoti hain Tool automatically resources create karta hai

Best for small tasks Best for large infrastructure & automation

Example: Azure CLI Example: Terraform

```
az group create \
--name MyResourceGroup \
--location centralindia
```

```
az storage account create \
--name mystorageaccount12345 \
--resource-group MyResourceGroup \
--location centralindia \
--sku Standard_LRS \
--kind StorageV2

az storage account list --output table

az storage account delete \
--name mystorageaccount12345 \
--resource-group MyResourceGroup
```

Connects to Azure

|

▼

Creates Infrastructure

Terraform Project Structure

Terraform project ko organize rakhne ke liye proper folder structure use kiya jata hai.

Example:

DevOps-Insiders

|

└── Terraform

|

└── rg-using-terraform

|

└── main.tf

Yahi folder hamara Terraform Project hota hai.

Terraform Configuration File

Terraform sirf .tf extension wali files ko hi read karta hai.

Valid Examples:

main.tf

resourcegroup.tf

storage.tf

network.tf

terraform.tf

❌ Invalid Examples:

main.txt

terraform.doc

azure.json

What is Written Inside .tf File?

.tf file ke andar hum infrastructure define karte hain.

Example:

Resource Group create karna

Storage Account create karna

Virtual Machine create karna

Virtual Network create karna

Yani hum infrastructure ko declare karte hain.

Isliye Terraform ko Declarative Tool kaha jata hai.

Terraform Installation Verification

Terraform install hone ke baad verify karne ke liye command run karte hain:

terraform --help

Agar help screen aa jaye to Terraform successfully install ho gaya hai.

Working in Project Folder

PowerShell ya Terminal me project folder open karo.

Example:

cd DevOps-Insiders

cd Terraform

cd rg-using-terraform

Ab jitni bhi Terraform commands chalengi, isi folder ke andar execute hongi.

Terraform Registry

Terraform ki official documentation ko Terraform Registry kehte hain.

Yahan se hume milta hai:

Providers

Resource documentation

Configuration examples

Sample code

Terraform Registry is the official reference for writing Terraform code.

Why Terraform Registry is Important?

Har resource ka syntax yaad rakhna zaroori nahi hota.

Developers generally:

Official documentation dekhte hain.

Example copy karte hain.

Apni requirement ke according modify karte hain.

Isi approach ko Documentation-Driven Development kaha jata hai.

Terraform Workflow

Terraform ka complete workflow bahut simple hai.

Step 1

Project Folder Create karo.

↓

Step 2

.tf file create karo.

↓

Step 3

Infrastructure code likho.

↓

Step 4

Terraform configuration execute karo.

↓

Step 5

Terraform Azure me infrastructure create kar dega.

Workflow Diagram

Create Project Folder

|

▼

Create main.tf

|

▼

Write Terraform Code

|

▼

Run Terraform Commands

|

▼

Terraform Creates Infrastructure

- Providers ki documentation dekh sakte ho.
- Resource syntax samajh sakte ho.
- Example code copy kar sakte ho.
- Modules use kar sakte ho.

Jab bhi code chahiye ho, Registry ya Google search use karo.

Common Searches

```
azure resource group terraform
aws s3 terraform
azure storage account terraform
azure virtual machine terraform
```

Terraform Provider

Provider Kya Hota Hai?

Provider Terraform aur Cloud Platform ke beech ek bridge (plugin) ki tarah kaam karta hai.

Ye Terraform ko batata hai ki kis cloud ke resources manage karne hain.

Common Providers

- azurerm → Microsoft Azure
- aws → Amazon Web Services
- google (gcp) → Google Cloud Platform

Example

```
provider "azurerm" {
    features {}
}
```

Yahan `azurerm` provider use karke Terraform Azure ke resources manage karega

Terraform Arguments

Arguments Kya Hote Hain?

Arguments wo inputs hote hain jo resource create karte waqt dene padte hain. Example: Agar Resource Group banana hai to uska name aur location dena padega.

Types of Arguments

1. Required Arguments

Ye mandatory hote hain. Inke bina resource create nahi hoga.

Examples

name

location

resource_group_name

Example: resource "azurerm_resource_group" "rg" {

name = "MyRG"

location = "Central India" }

Yahan name aur location required arguments hain.

2. Optional Arguments

Ye optional settings hoti hain. Inhe dena zaruri nahi hota.

Examples

tags

replication_type

account_tier

Example: tags = {

Environment = "Dev"

}

Agar tags nahi doge tab bhi resource create ho jayega.

Terraform Attributes

Attributes Kya Hote Hain?

Attributes wo values hoti hain jo resource create hone ke baad automatically generate hoti hain.

Yeh Terraform output ke through use ki ja sakti hain.

Examples

Resource ID

Endpoint

URL

IP Address

Name (generated or returned)

Example:

Jab Azure me Resource Group create hota hai to uski ek unique Resource ID generate hoti hai. Ye ek attribute hai.

Difference

Arguments → Input (jo hum dete hain)

Attributes → Output (jo Terraform/Cloud hume return karta hai)

Versioning in IT

Software ki nayi versions release karne ke 3 main reasons hote hain:

1. Security Fixes

Purani vulnerabilities aur security issues ko fix karna.

2. New Features

Naye features aur capabilities add karna.

3. Bug Fixes

Existing bugs aur errors ko solve karna

Terraform Commands

Terraform me available commands dekhne ke liye help command use karte hain.

Command

terraform --help

Ye command Terraform ke saare available commands aur unka basic usage dikha deti hai.

Common Commands

Command	Purpose
terraform init	Initialize the Terraform project
terraform validate	Validate configuration files
terraform plan	Show execution plan
terraform apply	Create or update infrastructure
terraform destroy	Delete infrastructure
terraform fmt	Format Terraform code
terraform version	Display installed Terraform version
terraform --help	Show all available command